SpaceNet Off-Nadir Building Segmentation

Dataset Preparation and PyTorch Input Pipeline
What we did, how we did it, and why — in simple terms

1. Purpose of This Document

This document records the work completed before the three pretrained segmentation models were trained. The project studies building-footprint segmentation on satellite images captured from different viewing angles. The goal of this stage was to build a clean, verified dataset and a PyTorch input pipeline so that the models can later receive the correct images and building masks.

Simple idea: We prepared and checked everything first. Only after the data pipeline worked correctly did we move toward model setup.

2. Overall Workflow

Stage	What we did	Why
1	Obtained SpaceNet imagery	Get the satellite images.
2	Obtained polygon annotations	Get building boundaries as ground truth.
3	Created building masks	Turn polygons into pixel-level labels.
4	Selected 20 valid geographic tiles	Use scenes with the required imagery.
5	Verified six viewing angles	Support angle-wise robustness evaluation.
6	Built 120 image instances	20 tiles × 6 angles.
7	Created geographic train/validation/test split	Prevent the same geographic scene appearing in different splits.
8	Checked dataset integrity	Catch missing files and incorrect dimensions before training.
9	Built PyTorch Dataset/DataLoaders	Convert files into tensors for neural networks.
10	Visually checked image + mask	Confirm that the numerical pipeline also makes sense.

3. SpaceNet Data and Viewing Angles

The project uses SpaceNet 4 / the SpaceNet Off-Nadir Building Detection data. The working dataset contains multiple views of the same geographic scenes at different satellite viewing angles. Six angle collections were used: **7°, 10°, 29°, 39°, 47°, and 53°**.

Why? The project is about robustness to off-nadir imagery. A building can appear different when the viewing angle changes because of perspective, roof displacement, shadows, and occlusion. Multiple views let us later measure performance separately at each angle.

4. Obtaining the Building Annotations

The downloaded **summaryData.tar.gz** was extracted into the project directory. It contained CSV annotation files. The important fields included ImageId, BuildingId, PolygonWKT_Pix, and PolygonWKT_Geo.

PolygonWKT_Pix contains building polygons in pixel coordinates. These polygons were rasterized to make binary building masks.

Why? Semantic segmentation needs a label for each pixel. The mask tells the model which pixels belong to buildings and which belong to background.

5. Finding Valid Image Tiles

A key dataset issue was discovered: an ImageId present in an annotation CSV did not necessarily mean that the corresponding sample TIFF existed in the downloaded imagery. One initially selected tile was present in the annotations but did not have the required 7° TIFF.

To avoid missing image/angle combinations, the actual imagery available in S3 was checked first. The candidate tiles were then checked across all six angle collections. The final set contained 20 tiles that existed at every required angle.

Why? A tile must have all six views if we want to compare the same geographic scene across the six viewing angles.

6. Final Dataset Size

Item	Value
Geographic tiles	20
Viewing angles	6: 7°, 10°, 29°, 39°, 47°, 53°
Image instances	120 (20 × 6)
Image size	900 × 900 pixels
Original TIFF bands	4
Buildings per angle	1,113
Polygon instances	6,678 (1,113 × 6)

The building count remains the same across angle views because the views correspond to the same geographic buildings. This does not mean the buildings look identical.

7. Dataset Analysis

The selected tiles were checked for building density and geographic spread. Building counts ranged from 1 to 159 per tile, with a mean of 55.65 and median of 45.

Building polygon areas were also examined. Across 6,678 polygon instances, the mean area was about 1,047 px², median about 525 px², and maximum about 52,256 px². The distribution was strongly right-skewed because a small number of large buildings occupied much more area than typical buildings.

Sparse tiles were not automatically removed. They were kept to preserve scene diversity, while recognizing that sparse test scenes can make final test metrics more variable.

8. Geographic Train / Validation / Test Split

The split was made at the **geographic tile level**. All six views of one tile stay in the same split. This prevents a model from seeing one angle of a location during training and another angle of the same location during testing.

Split	Tiles	Images	Buildings per angle
Train	14	84	866
Validation	3	18	126
Test	3	18	121
Total	20	120	1,113

Limitation: the test split contains two very sparse tiles (1 and 2 buildings) and one dense tile, so test estimates can have relatively high variance. The split is tile-level geographic separation, not a large strict spatial-block split.

9. Building the Final Dataset

The final dataset-building script read the split information, downloaded only the selected 20 tiles from the six required S3 collections, created masks from the polygon annotations, and wrote a metadata CSV for every image.

The original 4-band Pan-Sharpen TIFFs were preserved. Binary masks were stored as PNGs. Metadata recorded the tile, angle, category, split, image path, mask path, building count, image dimensions, and number of bands.

Why preserve the 4-band TIFFs? The first pretrained-model experiments use RGB, but preserving the original data keeps the fourth band available for a possible future experiment.

10. Final Dataset Integrity Check

The final integrity check confirmed: 120 metadata rows; train/validation/test counts of 84/18/18; 20 images at each of the six angles; every tile has all six angles; all image files exist; all mask files exist; all images are 900 × 900 × 4; and all masks are valid 900 × 900 binary masks.

This check is important because missing or mismatched images and masks could produce incorrect or misleading training results.

11. PyTorch Installation

PyTorch was installed to provide the framework for tensors, datasets, DataLoaders, neural networks, losses, optimizers, and training.

The computer did not have an NVIDIA GPU, so the CPU build was installed. This is sufficient for developing and testing the pipeline, although segmentation training will be slower than on a suitable GPU.

Verified: PyTorch 2.14.0+cpu; CUDA available = False; device = CPU.

12. Inspecting the TIFF

A real dataset TIFF was inspected with Rasterio. It was confirmed to be 900 × 900 pixels, contain four uint16 bands, and have no internal band descriptions.

For the initial pretrained RGB models, the WorldView-2 four-band product was interpreted as B, G, R, NIR, so the first three relevant bands were reordered to R, G, B for the model input. The original four-band TIFF remains untouched.

13. Creating the PyTorch Dataset

A custom **SpaceNetDataset** class was created. For each metadata row it reads the TIFF and its mask, converts the image to RGB, converts it to float32, normalizes it for pipeline testing, converts the mask from 0/255 to 0/1, and returns the image and mask.

The temporary normalization used at this stage was per-image min-max scaling to 0–1. This was useful for testing the pipeline, but it was **not treated as the final preprocessing choice** for an ImageNet-pretrained ResNet50.

Resulting tensor format:

Image: [3, 900, 900]
Mask: [1, 900, 900]
3 = RGB channels; 1 = binary mask channel.

14. Creating DataLoaders

PyTorch DataLoaders were created for train, validation, and test sets. A batch size of 2 was used initially because the computer is CPU-only and the images are large.

The verified batch was:

Images: torch.Size([2, 3, 900, 900])
Masks: torch.Size([2, 1, 900, 900])
Mask values: tensor([0., 1.])

15. Visual Sanity Check

A batch was visualized by displaying an RGB satellite image next to its ground-truth building mask. The image and mask showed the expected spatial relationship, confirming that the pipeline was not simply producing correct tensor shapes but was also pairing images and labels sensibly.

The RGB display looked relatively dark. This was not treated as a dataset failure because displayed brightness depends on the current normalization. Final model preprocessing was left for the model-specific stage.

16. DeepLabV3+ Compatibility Check

After the general data-preparation stage, the assigned DeepLabV3+ + ResNet50 model was connected to the DataLoader for a forward-pass test. DeepLabV3+ required image dimensions divisible by 16, so the 900 × 900 input was temporarily padded to 912 × 912.

The model output was then cropped back to 900 × 900. The successful test produced: original input [2, 3, 900, 900], padded input [2, 3, 912, 912], and model output [2, 1, 900, 900].

This confirmed that the prepared dataset can be passed through the model without changing the stored source images.

17. Completed Checklist

- ✓ SpaceNet imagery obtained
- ✓ Polygon annotations obtained
- ✓ Building masks generated
- ✓ 20 geographic tiles selected
- ✓ Six viewing angles verified
- ✓ 120 image instances created
- ✓ Geographic train/validation/test split
- ✓ Dataset integrity checked
- ✓ 4-band TIFFs preserved
- ✓ PyTorch installed

- ✓ PyTorch Dataset created
- ✓ RGB conversion implemented
- ✓ Binary masks implemented
- ✓ DataLoaders created
- ✓ Batch dimensions verified
- ✓ Image + mask visually verified

18. Where We Stop

At the end of this stage, the project has a verified 120-image dataset and a working PyTorch input pipeline. The next stage is model-specific work. The assigned model is DeepLabV3+ with a ResNet50 encoder and ImageNet-pretrained weights.

Important: the final training normalization, loss function, optimizer, training loop, checkpointing, and angle-wise evaluation belong to the next stage and are not part of this data-preparation document.